```
void trocar(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

void inverterSubcadeia(char str[], int inicio, int fim) {
    while (inicio < fim) {
        char temp = str[inicio];
        str[fim] = temp;
        inicio++;
        fim--;
    }
}

int removeRepetidos(int v[], int tam) {
    if (tam <= 0) return 0;
    int k = 0;
    for (int i = 1; i < tam; i++) {
        if (v[i] != v[k]) {
            k++;
            v[k] = v[i];
        }
    }
    return k + 1;
}
```

Credeal

// //

```
void ordenar (int v[], int tam) {
    for (int i=0; i < tam; i++) {
        for (int j=0; j < tam - 1; j++) {
            if (v[j] > v[j+1]) {
                trocar (&v[j], &v[j+1]);
            }
        }
    }
}
```

- (quantidade incorreta e incoerente

"poderiam ser mais refinadas p/ evitar redundância"

```
void preencherPrimos (int v[], int tam) {
    int cont = 0;
    int candidato = 2;
    while (cont < tam) {
        int umPrimo = 1;
        for (int i=0; i < cont; i++) {
            if (candidato % v[i] == 0) {
                umPrimo = 0;
            }
        }
        if (umPrimo == 1) {
            v[cont] = candidato;
            cont++;
        }
        candidato++
    }
```

falta especificar de usar break;

deal

```
void maiorPorLinha (int m[][QTD_COLUNAS], int lin, int col, int v[]) {
    for (int i = 0; i < lin; i++) {
        int maior = 0;                    → int maior = m[i][0]
        for (int j = 0; j < col; j++) {
            if (m[i][j] > maior) {
                maior = m[i][j];
            }
        }
        v[i] = maior;
    }
}
```

Se a matriz conter apenas números negativos cairá incorretamente

```
void inverterPalavras (char str[]) {
    int inicio = 0;
    int i = 0;
    while (str[i] != '\0') {
        if (str[i] == ' ') {
            inverterSubcadeia (str, inicio, i-1);
            inicio = i + 1;
        }
        i++;
    }
    inverterSubcadeia (str, inicio, i-1);
}
```

boa lógica
tratamento errado